Documentación de Pruebas — Sistema de Lavado Vehicular

Portada

Tabla 1

Datos de identificación del documento

Campo	Valor
Proyecto	Sistema de Gestión de Lavado Vehicular (Vehicle Washing / LavaRápido)
Producto	Documentación de pruebas (sección 2.5.4)
Programa	Tecnólogo en Análisis y Desarrollo de Software (ADSO)
Ficha	3145556
Aprendices	Brayan Estiven Patiño Cabrera, Juan Camilo Guilombo Osorio, Merari Urbano
Instructora	Karol Daniela Correa
Centro de formación	Industria
Fecha	16 de octubre de 2026

Este documento reúne el plan de pruebas, el inventario de pruebas automatizadas que existen en los repositorios, sus resultados reales al ejecutarlas el 16 de octubre de 2026 (incluidas las pruebas de punta a punta contra el sistema completo) y los casos de prueba funcionales, ejecutados el 9 de octubre de 2026, trazados a los requisitos del SRS v7 (`lavarapido-docs/01_INFORME_DE_ESPECIFICACION_DE_REQUISITOS`). La estructura sigue la norma ISO/IEC/IEEE 29119-3 de documentación de pruebas (International Organization for Standardization [ISO], 2021). Ningún resultado de este documento es supuesto.

1. Plan de pruebas

1.1 Alcance

Se prueban la aplicación web, la aplicación móvil y los siete servicios de backend en su rama `develop`. Quedan fuera las pruebas de seguridad ofensiva; la carga se revisa solo con la medición de tiempos de respuesta de RNF-001.

1.2 Tipos de prueba y herramientas

Tabla 2

Tipos de prueba, herramienta real y dónde se aplica

Tipo	Qué verifica	Herramienta	Dónde
Unitarias de dominio	Reglas de entidades y objetos valor (estados, validaciones)	JUnit 5 (`spring-boot-starter-test`); xUnit 2	`src/test/.../domain` en Java; `tests/PaymentService.Domain.UnitTests`
Unitarias de aplicación	Casos de uso con repositorios en memoria (dobles de prueba)	JUnit 5 con fakes (`Fakes.java`, `InMemory*Repository`); xUnit con `InMemoryPorts.cs`	`application/usecase` y `PaymentService.Application.UnitTests`
Integración web (controlador + seguridad)	Rutas, códigos HTTP, JWT y roles con MockMvc	`spring-boot-starter-webmvc-test`, `spring-security-test`	`*ApiWebTest.java`, `ApiSecurityWebTest.java`, `AdminNotificationControllerTest.java`
Integración de rutas del gateway	Que cada ruta pública llegue a su servicio	Spring Boot Test	`GatewayRoutesTest.java`
Integración de persistencia	Mapeo EF Core contra SQL Server real	xUnit + `DatabaseFixture`	`PaymentService.Infrastructure.IntegrationTests` (requiere base de datos)
Unitarias de interfaz web	Componentes, guards y servicios de Angular	Vitest 4 + jsdom (`@angular/build:unit-test`)	`Web/src/**/*.spec.ts`

Tipo	Qué verifica	Herramienta	Dónde
Funcionales / aceptación	Recorridos completos por rol contra el sistema desplegado	Manual, con Postman o Swagger UI para la API y navegador / Expo Go para la interfaz	Casos CP-01 a CP-33 (sección 4)
Entre servicios (punta a punta) y RNF	Reserva con elección de operario y asignación automática, flujo completo de los tres roles, informe de inspección, reglas de acceso por rol; percentil 95 del tiempo de respuesta (RNF-001); pantallas web a 360 px (RNF-008)	Node 24 (`node:test` y `fetch`) y Playwright Core con Edge sin ventana	`lavarapido-infra/e2e`

> *Nota.* Herramientas documentadas en JUnit (s.f.), xUnit.net (s.f.) y Vitest (s.f.).

1.3 Entorno

Tabla 3

Entorno de ejecución de las pruebas automatizadas

Elemento	Versión
Sistema operativo	Windows 11 Home Single Language 10.0.26300 (15 y 16 de octubre); 10.0.26200 (9 de octubre)
Node.js / npm	24.14.0 / 11.9.0
JDK	Java 25 (los proyectos compilan para Java 21) con el Maven Wrapper de cada servicio
.NET SDK	9.0.306; el proyecto apunta a `net8.0` y las pruebas se corrieron con `DOTNET_ROLL_FORWARD=Major`
Base de datos y mensajería	Las pruebas unitarias no las necesitan. Las pruebas de punta a punta del 16 de octubre y los casos funcionales del 9 de octubre usaron SQL Server 2022, RabbitMQ 4 y Mailpit en Docker 28.4 (Docker Desktop)
Servicios	Para los casos funcionales, los 7 servicios en el equipo (JAR de Spring Boot y `dotnet run`); el correo se redirigió a Mailpit para no enviar correos reales

1.4 Criterios

- **Aprobado:** el resultado obtenido coincide con el esperado.
- **Fallido:** el resultado difiere; se registra el error.
- **Bloqueado:** no se pudo ejecutar por una dependencia ausente (por ejemplo la base de datos).
- **Salida:** se acepta la versión si las suites automatizadas pasan y los casos funcionales de prioridad alta están aprobados.

2. Inventario de pruebas automatizadas

Los repositorios tienen 542 métodos de prueba: 499 en el backend y 43 en la web. La app móvil no tiene pruebas automatizadas.

Tabla 4

Pruebas por artefacto (conteo de `@Test`/`@ParameterizedTest`, `[Fact]`/`[Theory]`/`[IntegrationFact]` e `it()`)

Artefacto	Archivos	Métodos	Qué cubren
api-gateway	1	8	Ruteo de cada prefijo `/api/v1/**` a su servicio y que `/internal/**` no salga por el gateway (`GatewayRoutesTest`)
security-service	17	131	Registro, login con bloqueo por intentos fallidos, recuperación de contraseña, perfil y dirección, administración de usuarios, roles y permisos, preferencias, objetos valor, política de contraseñas, emisión de JWT y seguridad HTTP por rol (`ApiSecurityWebTest`)
customer-service	6	50	Cuenta del cliente, placas, CRUD de vehículos, listener de `security.user_registered` y API web

Artefacto	Archivos	Métodos	Qué cubren
booking-service	6	40	Ciclo de vida de la reserva, disponibilidad, catálogo, datos del negocio (`EstablishmentTest`), operario preferido en el evento y API web
operations-service	11	54	Asignación manual y automática, operarios libres, ventana de inicio, ejecución, calificación, métricas del operario, resumen del equipo, ficha del operario, informe de inspección y API web
notification-service	12	106	Casos de uso de notificaciones, fábrica de mensajes por evento, avisos en el idioma de la cuenta, recordatorios, alertas de reservas sin operario, tokens de dispositivo y controlador admin
payment-service (.NET)	9	110	Pago y comprobante, referencia reutilizada, historial con filtros, promociones, puntos de fidelización, política de pagos, servicios de comandos y consultas, y 7 pruebas de persistencia contra SQL Server
Web (Angular)	37	43	Creación de componentes, guards y servicios (pruebas de humo `should create`), mensajes de error de la API y selección de la cuenta de pago
Móvil	0	0	Sin pruebas automatizadas

> *Nota.* Los archivos de apoyo (`Fakes.java`, `InMemory*.java`, `InMemoryPorts.cs`, `DatabaseFixture.cs`) no se cuentan como pruebas. Las 7 pruebas de persistencia de payment se omiten solas si no está definida la variable de conexión a SQL Server (`DatabaseFixture.cs:20-21`).

Las pruebas del backend son de dominio y de aplicación con dobles en memoria, más pruebas de controladores con MockMvc. La mayoría de las de la web comprueban que cada componente se crea; `api-error.spec.ts` y `payment.spec.ts` prueban reglas reales (los mensajes de error de la API y la selección de la cuenta de pago).

3. Resultados de ejecución

El 16 de octubre de 2026 se ejecutaron las ocho suites automatizadas: 639 pruebas, todas aprobadas. Las 7 de persistencia de payment se omitieron porque no se definió la base de pruebas; el 9 de octubre se habían ejecutado contra una base aislada (`LavaRapidoPaymentIT`) creada con los mismos changelogs y pasaron.

Tabla 5

Resultados reales por artefacto (16 de octubre de 2026)

Artefacto	Comando	Ejecutadas	Aprobadas	Fallidas	Omitidas	Duración
Web (Angular)	`npx ng test --watch=false`	43	43	0	0	6,7 s
api-gateway	`./mvnw test`	34	34	0	0	9,9 s
security-service	`./mvnw test`	147	147	0	0	13,5 s
customer-service	`./mvnw test`	68	68	0	0	8,3 s
booking-service	`./mvnw test`	40	40	0	0	10,0 s
operations-service	`./mvnw test`	54	54	0	0	10,9 s
notification-service	`./mvnw test`	112	112	0	0	0,6 s
payment-service: dominio	`dotnet test PaymentService.sln`	113	113	0	0	0,1 s
payment-service: aplicación	(mismo comando)	28	28	0	0	0,1 s
payment-service: integración	(mismo comando, sin `PAYMENT_INTEGRATION_DB`)	0	0	0	7	—
Total		**639**	**639**	**0**	**7**	

> *Nota.* Fuentes: salida de Vitest, archivos `target/surefire-reports/*.xml` de cada servicio Java y salida de `dotnet test`. Las duraciones de Maven suman el tiempo de las clases de prueba, sin contar la compilación. Hay más pruebas ejecutadas que métodos en la Tabla 4 porque las pruebas parametrizadas (`@ParameterizedTest`, `[Theory]`) corren una vez por cada dato.

Observaciones de la ejecución:

- La línea `ERROR ... Could not send the password recovery email: SMTP down` del log de security-service la provoca a propósito `EmailResetCodeNotifierTest`, que simula un SMTP caído. No es una falla.
- Vitest muestra un aviso de compilación: `src/test-providers.ts` no está en el `tsconfig`. No afecta los resultados.
- La ejecución usó JDK 25 y .NET SDK 9 sobre proyectos para Java 21 y .NET 8; para la entrega conviene repetirla con las versiones exactas del proyecto.

3.1 Pruebas de punta a punta

El 16 de octubre de 2026 se ejecutó `node --test "e2e/*.test.js"` desde `lavarapido-infra`, con los contenedores del perfil *app* en marcha y `E2E_OPERATOR_ID=2` (el operario de prueba de la base local). Pasaron las 19 pruebas en 7,3 s. Cada prueba borra lo que crea y deja los puntos del cliente como estaban.

Tabla 5a

Pruebas de punta a punta (16 de octubre de 2026)

Archivo	Pruebas	Qué recorre	Resultado
`access-rules.test.js`	7	Permisos por rol y rutas internas, dirección del perfil, filtros del historial de pagos, datos tributarios solo para el administrador, ficha del operario, hora fuera del horario (`reason: OUTSIDE_HOURS`) y token de una cuenta inexistente	7 aprobadas
`booking-flow.test.js`	5 (1 + 4 subpruebas)	Reserva eligiendo operario, asignación automática por RabbitMQ, avisos al cliente, resumen del día y rutas internas fuera del gateway	5 aprobadas
`full-flow.test.js`	6 (1 + 5 subpruebas)	Reserva → pago reportado → aprobación con puntos → inicio y fin del servicio → calificación, revisando cada aviso	6 aprobadas
`inspection-flow.test.js`	1	Informe de inspección con hallazgos y fotos, publicación, aviso al cliente y enlace público	1 aprobada
Total	**19**		**19 aprobadas**

4. Casos de prueba funcionales

Los 33 casos cubren los requisitos que estaban implementados o parciales en el SRS v4. Se ejecutaron el 9 de octubre de 2026 entre las 15:40 y las 15:45 contra el sistema completo, a través del API Gateway (las mismas llamadas que hacen la web y el móvil), con un script automatizado. Los 33 resultaron aprobados. Los requisitos que quedaron implementados después (RF-004b, RF-008, RF-010, RF-015, RF-022, RF-036, RF-037 y RF-038) están cubiertos por pruebas automatizadas (sección 5), y el flujo de reserva con elección de operario y asignación automática lo recorre además `lavarapido-infra/e2e/booking-flow.test.js`.

Tabla 6

Casos de prueba funcionales

ID	RF	Caso	Precondición	Pasos	Datos	Resultado esperado	Resultado obtenido	Estado
CP-01	RF-001	Registro de cliente exitoso	Correo no registrado	Registrar la cuenta e iniciar sesión	Documento, nombre, teléfono, correo nuevo y contraseña válida	Cuenta creada con rol CLIENT; se crea el perfil en customer-service	Registro 201; login con rol CLIENT; perfil de cliente disponible (`GET /vehicles` 200)	Aprobado
CP-02	RF-001	Registro con correo repetido	El correo ya existe	Registrar con el mismo correo	Correo de CP-01	Mensaje de correo ya registrado; no se crea la cuenta	409 `EMAIL_ALREADY_REGISTERED`; no se creó la cuenta	Aprobado

ID	RF	Caso	Precondición	Pasos	Datos	Resultado esperado	Resultado obtenido	Estado
CP-03	RF-002	Inicio de sesión por rol	Usuarios CLIENT, OPERATOR y ADMIN	Iniciar sesión con cada uno	Credenciales válidas	Redirige a `/client`, `/operator` o `/admin` según el rol	Los tres logins devuelven JWT con roles ADMIN, OPERATOR y CLIENT; la web redirige por ese rol (`authGuard`)	Aprobado
CP-04	RF-002	Credenciales incorrectas	—	Iniciar sesión con contraseña errónea y luego con correo inexistente	Dos intentos	Mismo mensaje genérico en ambos; no revela si el correo existe	Ambos: 401 `INVALID_CREDENTIALS` (misma respuesta)	Aprobado
CP-05	RF-003	Recuperar contraseña	Usuario registrado; Mailpit activo	Pedir código, leerlo en Mailpit, verificarlo y fijar nueva contraseña	Código de 6 dígitos	Contraseña cambiada; el login con la nueva funciona	Código de 6 dígitos recibido en Mailpit; verify 204, reset 204; login con la nueva contraseña exitoso	Aprobado
CP-06	RF-003	Código bloqueado por intentos	Código solicitado	Ingresar 5 códigos erróneos y luego el correcto	5 códigos inválidos	El código correcto ya no es aceptado	5 intentos: 400; luego el código correcto: 400 `INVALID_RESET_CODE`	Aprobado
CP-07	RF-004	Cambio de contraseña con actual incorrecta	Sesión iniciada	Cambiar contraseña con la actual equivocada	Contraseña actual errónea	Error de contraseña actual incorrecta; no cambia	400 `INCORRECT_CURRENT_PASSWORD`; el login con la contraseña actual sigue funcionando	Aprobado
CP-08	RF-005	Crear reserva	Cliente con vehículo SUV	Elegir vehículo, servicio Básico, fecha y hora libre	SUV, Básico	Reserva creada; precio $23.000 (precio semilla de Básico para SUV)	Reserva 201 RES-060003 para el 10/10/2026 08:00, estado CONFIRMED; total $23.000	Aprobado
CP-09	RF-006	Horario ocupado	Bahías activas de una franja ocupadas	Reservar en la misma franja hasta llenarla	Varios vehículos del cliente	No se guarda la reserva; se ofrecen hasta cinco horas alternativas del mismo día	Franja 08:30 llena (BAY-02 y BAY-03 más una reserva previa); la siguiente: 409 `SLOT_UNAVAILABLE` con alternativas 09:00, 09:30, 10:00, 10:30 y 11:00	Aprobado
CP-10	RF-007	Cancelar reserva antes de iniciar	Reserva CONFIRMED	Cancelar con motivo	Motivo `CUSTOMER_REQUEST`	Reserva en CANCELLED con el motivo guardado	200: estado CANCELLED, motivo `CUSTOMER_REQUEST`	Aprobado
CP-11	RF-007	Modificar reserva en proceso	Reserva IN_PROGRESS	Intentar cancelar	—	Acción bloqueada con mensaje	Reserva IN_PROGRESS; cancelar: 409 `BOOKING_NOT_CHANGEABLE`	Aprobado
CP-12	RF-009	Asignar operario no disponible	Operario con ausencia a la hora de la reserva	Consultar candidatos e intentar asignarlo; luego asignar uno disponible	Operario ausente	El operario no se permite; los disponibles aparecen primero	Disponibles primero; el ausente aparece con motivo `OPERATOR_ABSENT`; asignarlo: 409 `OPERATOR_ABSENT`; asignar al disponible: 200	Aprobado
CP-13	RF-011	Operario ve sus servicios	Reserva asignada al operario	Consultar servicios asignados	—	Lista con datos del cliente y del vehículo	RES-060003 aparece con vehículo Mazda CX-5, placa FPW-653 y servicio Básico	Aprobado
CP-14	RF-012b	Iniciar y finalizar servicio	Servicio asignado PENDING, a 15 min de su hora	Otro operario intenta iniciar; el asignado inicia y finaliza	—	Estados IN_PROGRESS y COMPLETED; el otro operario recibe error	Otro operario: 404 `BOOKING_NOT_FOUND`; iniciar 200 (IN_PROGRESS), finalizar 200 (COMPLETED); reserva COMPLETED	Aprobado
CP-15	RF-012	Calificar servicio una vez	Reserva COMPLETED	Calificar 5 estrellas; intentar calificar de nuevo	5, comentario	Primera calificación guardada; la segunda es rechazada	Primera: 201 (5); segunda: 409 `ALREADY_RATED`	Aprobado

ID	RF	Caso	Precondición	Pasos	Datos	Resultado esperado	Resultado obtenido	Estado
CP-16	RF-013	Métodos de pago	Cuentas de pago configuradas	Consultar métodos (admin) y cuentas (cliente)	—	Efectivo, Nequi, Daviplata y Transferencia; el efectivo sin QR	El admin ve los 4 métodos; el API da al cliente solo las cuentas digitales activas con QR, y la interfaz agrega siempre el efectivo sin QR (`payment.ts:44-57`)	Aprobado
CP-17	RF-013b	Subir comprobante	Reserva por pagar	Pagar con Nequi adjuntando imagen, valor y referencia	PNG < 10 MB	Pago creado en revisión con su comprobante	201: estado IN_REVIEW, monto $24.000, referencia REF-428646	Aprobado
CP-18	RF-014	Aprobar y rechazar pago	Pagos en revisión	Aprobar uno y rechazar otro con motivo	Motivo de rechazo	Estados APPROVED y REJECTED; el cliente recibe notificación	Aprobar 200 (APPROVED); rechazar 200 (REJECTED); el cliente recibió `PAYMENT_CONFIRMED` y `PAYMENT_REJECTED`	Aprobado
CP-19	RF-016	Referencia repetida	Pago aprobado con referencia R1	Subir otro comprobante con R1 en otra reserva	Referencia R1	Comprobante rechazado	409 `TRANSACTION_REFERENCE_REUSED`	Aprobado
CP-20	RF-018	Notificación de reserva	`MESSAGING_ENABLED=true`	Confirmar una reserva; abrir notificaciones; marcar como leída	—	Llega la notificación; el contador de no leídas baja	9 notificaciones (`USER_WELCOME`, `BOOKING_CONFIRMED`, `BOOKING_CANCELLED`, `OPERATOR_ASSIGNED`); no leídas 9 → 8	Aprobado
CP-21	RF-020	Idioma guardado en la cuenta	Sesión iniciada	Cambiar a inglés; iniciar una sesión nueva	`en`	La interfaz abre en inglés	Preferencia guardada (200); la sesión nueva devuelve `language=en` y la web abrió en inglés (captura de prueba)	Aprobado
CP-22	RF-021	Excepción de horario	—	Crear excepción "cerrado" y consultar disponibilidad ese día	20/10/2026	No hay horas disponibles ese día	Excepción 201; disponibilidad: `open=false`, sin horas libres	Aprobado
CP-23	RF-024	Crear y desactivar promoción	Sesión de admin	Crear promoción de 10 % y desactivarla	Descuento 10 %	Promoción visible y luego inactiva	Creada 201; desactivada 200; estado en la lista: `paused`	Aprobado
CP-24	RF-026	Acumular puntos	Servicio con 10 puntos configurados	Aprobar el pago de la reserva; ver saldo	—	El saldo aumenta en los puntos del servicio	Saldo 0 → 10 al aprobar el pago	Aprobado
CP-25	RF-027	Desactivar la cuenta propia	Sesión de cliente	Desactivar la cuenta con la contraseña actual e intentar iniciar sesión	Contraseña actual	La sesión se revoca y el login se rechaza	Desactivación 204; nuevo login: 403 `ACCOUNT_DISABLED`	Aprobado
CP-26	RF-028	Crear operario y restringir por rol	Sesiones de admin y de cliente	Admin crea operario; el cliente llama `GET /admin/users`	Datos del operario	Cuenta OPERATOR creada; el cliente recibe 403	Operario creado 201; el cliente recibe 403	Aprobado
CP-27	RF-029	Editar la matriz de permisos	Sesión de admin	Quitar `EDIT_DATA` a OPERATOR; intentar crear un rol	Permiso `EDIT_DATA`	La matriz se guarda; no se pueden crear roles	Matriz guardada (200); crear rol: 405. Al final se restauró la matriz	Aprobado
CP-28	RF-030	Registrar vehículo con placa repetida	Cliente con un vehículo registrado	Registrar otro vehículo con la misma placa	Placa FPW653	El registro se rechaza	Primer registro 201; segundo: 409 `PLATE_ALREADY_REGISTERED`	Aprobado

ID	RF	Caso	Precondición	Pasos	Datos	Resultado esperado	Resultado obtenido	Estado
CP-29	RF-031	Cambio de precio sin afectar reservas	Reserva confirmada con Básico	Admin sube el precio de Básico/SUV; consultar la reserva existente y crear una nueva	$23.000 → $24.000	La existente conserva su precio; la nueva usa el nuevo	Reserva previa: $23.000; reserva nueva: $24.000. Al final se restauró el precio	Aprobado
CP-30	RF-032	Bahía en mantenimiento	BAY-04 en mantenimiento (semilla)	Llenar una franja y revisar qué bahías se usaron	—	La bahía en mantenimiento no se usa para reservas nuevas	BAY-04 no se usó; la franja se dio por llena solo con las bahías activas	Aprobado
CP-31	RF-033	Transición de estado no permitida	Reserva CANCELLED	Admin intenta cambiarla a CONFIRMED	—	Se rechaza	409 `INVALID_STATUS_TRANSITION`	Aprobado
CP-32	RF-034	Cuenta de pago desactivada	Cuenta Daviplata activa	Admin la desactiva; el cliente consulta cuentas	—	La cuenta ya no se muestra al cliente	Visible al crearla; desactivada (200) ya no aparece al cliente	Aprobado
CP-33	RF-035	Reembolsar un pago aprobado	Pago APPROVED	Admin reembolsa	—	Estado REFUNDED; se publica `payment.refunded` y el cliente es notificado	Reembolso 204; pago en REFUNDED; notificación `PAYMENT_REFUNDED` recibida; puntos revertidos (10 → 0)	Aprobado

Nota. Los precios y códigos de los datos de prueba salen de los changelogs semilla (`103-seed-catalog.sql`, `104-seed-booking-catalogs.sql`). Los casos CP-25 a CP-33 cubren los requisitos RF-027 a RF-035, nuevos en el SRS v4. Se usaron cuentas de prueba nuevas, porque la cuenta demo `operador@gmail.com` está desactivada en la base local. Al terminar se restauraron el precio de Básico, la matriz de permisos, la promoción y la excepción de horario. Evidencia: registro de ejecución con código HTTP y código de error de cada paso.

5. Matriz de trazabilidad

De los 41 requisitos del SRS v7, 30 tienen caso funcional y 39 una prueba automatizada relacionada; RF-032 y RF-034 solo tienen caso funcional.

Tabla 7

RF → estado → endpoint / pantalla → caso de prueba

RF	Estado (SRS v7)	Endpoint (prefijo `/api/v1`)	Pantalla web	Caso funcional	Prueba automatizada relacionada
RF-001	Implementado	`POST /auth/register`	`/auth/register`	CP-01, CP-02	`AccountRegistrationServiceTest`, `UserRegisteredListenerTest`
RF-002	Implementado	`POST /auth/login`	`/auth`	CP-03, CP-04	`AuthenticationServiceTest`, `JwtTokenIssuerTest`, `ApiSecurityWebTest`
RF-003	Implementado	`POST /auth/password/forgot`, `/verify`, `/reset`	`/auth/forgot-password`	CP-05, CP-06	`PasswordRecoveryServiceTest`, `PasswordResetTokenTest`
RF-004	Implementado	`PUT /users/me/password`	`/client/profile`	CP-07	`UserProfileAndAdministrationTest`, `PasswordPolicyTest`
RF-004b	Implementado	`POST /auth/login` (423 `ACCOUNT_LOCKED`)	`/auth`	—	`AuthenticationServiceTest` (bloqueo y ventana), `PasswordRecoveryServiceTest` (el restablecimiento lo levanta), `ApiSecurityWebTest` (423)

RF	Estado (SRS v7)	Endpoint (prefijo `/api/v1`)	Pantalla web	Caso funcional	Prueba automatizada relacionada
RF-027	Implementado	`GET/PATCH /users/me`, `GET/PUT /users/me/address`, `PUT /users/me/email`, `POST /auth/logout`, `POST /users/me/deactivate`	`/client/profile`	CP-25	`UserProfileAndAdministrationTest`, `PersonAddressServiceTest`, `AuthenticationServiceTest`
RF-028	Implementado	`GET/POST /admin/users`, `PATCH /admin/users/{id}/status`	`/admin/management`, `/admin/operators`	CP-26	`UserProfileAndAdministrationTest`, `ApiSecurityWebTest`
RF-029	Implementado	`GET /admin/roles-permissions`, `PUT /admin/roles-permissions/{role}`	`/admin/management`	CP-27	`RolePermissionAdministrationServiceTest`
RF-030	Implementado	`/vehicles`, `GET /vehicle-types`, `GET /admin/vehicles`	`/client/vehicles`	CP-28	`CustomerVehicleServiceTest`, `LicensePlateTest`, `CustomerApiWebTest`
RF-031	Implementado	`GET /catalog/services`, `/admin/catalog/services`	`/admin/management`	CP-29	`CatalogServiceTest`
RF-021	Implementado	`PUT /admin/schedule/business-hours`, `POST /admin/schedule/exceptions`	`/admin/schedule`	CP-22	`AvailabilityPolicyTest`
RF-032	Implementado	`/admin/bays`	`/admin/schedule`	CP-30	—
RF-037	Implementado	`GET/PUT /admin/establishment`, `GET /establishment`	`/admin/settings`	—	`EstablishmentTest`, `BookingApiWebTest`
RF-005	Implementado	`POST /bookings`, `GET /catalog/services`	`/client/reserve`	CP-08	`BookingServiceTest`, `BookingTest`
RF-006	Implementado	`GET /bookings/availability`	`/client/reserve`	CP-09	`AvailabilityPolicyTest`
RF-007	Implementado	`PUT /bookings/{id}`, `POST /bookings/{id}/cancel`	`/client/history`	CP-10, CP-11	`BookingTest`, `BookingApiWebTest`
RF-033	Implementado	`GET /admin/bookings`, `PATCH /admin/bookings/{id}/status`	`/admin/reservations`	CP-31	`BookingTest`, `BookingServiceTest`
RF-008	Implementado	Evento `booking.confirmed`, `GET /internal/v1/bookings`	`/admin/reservations`	—	`AutoAssignmentPolicyTest`, `OperationsServiceTest` (asignación automática, cuenta desactivada), `booking-flow.test.js`
RF-009	Implementado	`GET /admin/assignments/{id}/candidates`, `PUT /admin/assignments/{id}`	`/admin/reservations`	CP-12	`AssignmentPolicyTest`, `OperationsServiceTest`
RF-010	Implementado	`GET /operators/available`, `POST /bookings` (`preferredOperatorId`)	`/client/reserve`	—	`OperationsServiceTest` (operarios libres), `OperationsApiWebTest`, `booking-flow.test.js`
RF-011	Implementado	`GET /operator/services`	`/operator/assigned-services`	CP-13	`OperationsServiceTest`
RF-012b	Implementado	`POST /operator/services/{id}/start`, `/finish`	`/operator/assigned-services`	CP-14	`StartPolicyTest`, `ServiceExecutionTest`
RF-012	Implementado	`POST /ratings/{bookingId}`	`/client/history`	CP-15	`ServiceExecutionTest`
RF-022	Implementado	`PUT /admin/operators/{id}/availability`, `/absences`, `GET /admin/operators/summary`	`/admin/operators/:id`	—	`AssignmentPolicyTest`, `TeamSummaryCalculatorTest`
RF-036	Implementado	`/admin/inspections/{bookingId}/**`, `GET /public/inspections/{token}`	`/admin/reservations/:id/inspection`, `/report/:token`	—	`InspectionReportTest`, `OperationsApiWebTest`
RF-038	Implementado	`GET /admin/operators/profiles`, `PUT /admin/operators/{id}/profile`	`/admin/operators/:id`	—	`OperatorProfileServiceTest`
RF-034	Implementado	`/admin/payment-accounts`, `GET /payment-accounts`	`/admin/settings`	CP-32	—
RF-013	Implementado	`GET /payment-accounts`	`/client/payment`	CP-16	`PaymentQueryServiceTests`
RF-013b	Implementado	`POST /payments`	`/client/payment`	CP-17	`PaymentTests`, `PaymentCommandServiceTests`
RF-014	Implementado	`POST /admin/payments/{id}/approve`, `/reject`	`/admin/payments`	CP-18	`PaymentTests`, `PaymentCommandServiceTests`
RF-015	Implementado	`GET /payments/me?from=&to=&service=`	`/client/history`	—	`PaymentQueryServiceTests` (orden y filtros)
RF-016	Implementado	`POST /payments`	`/client/payment`	CP-19	`PaymentPolicyTests`, `PaymentCommandServiceTests` (referencia reutilizada)

RF	Estado (SRS v7)	Endpoint (prefijo `/api/v1`)	Pantalla web	Caso funcional	Prueba automatizada relacionada
RF-035	Implementado	`POST /admin/payments/{id}/refund`	`/admin/payments`	CP-33	`PaymentTests`, `PaymentCommandServiceTests`
RF-017	Implementado	`GET /admin/bookings/day-summary`, `GET /operators/my-bookings` y stores del dashboard	`/client`, `/operator`, `/admin`	—	`BookingServiceTest` (resumen del día), pruebas de humo de la web (`dashboard.spec.ts`)
RF-018	Implementado	`/notifications/*` + eventos RabbitMQ	`/*/notifications`	CP-20	`NotificationUseCasesTest`, `NotificationFactoryTest`, `BookingReminderServiceTest`, `LocalizedNotificationsTest`, `UnassignedBookingAlertTest`, `StaffNotificationsTest`
RF-019	Implementado	`PUT /users/me/preferences`	`/*/configuration`	Cubierto por CP-21	`UserPreferenceServiceTest`
RF-020	Implementado	`PUT /users/me/preferences`	`/*/configuration`	CP-21	`UserPreferenceServiceTest`, `UserPreferenceTest`
RF-023	Parcial	`GET /admin/operators/{id}/metrics`, `GET /admin/operators/summary`, stores de pagos y reservas	`/admin/reports`, `/admin/operators/:id`	—	`OperatorMetricsCalculatorTest`, `TeamSummaryCalculatorTest`
RF-024	Implementado	`/admin/promotions`	`/admin/management`	CP-23	`PromotionTests`
RF-025	Implementado	`POST /loyalty/redeem`	`/client/payment`	—	`CustomerPromotionServiceTests`
RF-026	Implementado	`GET /loyalty/balance`	`/client`	CP-24	`LoyaltyTransactionTests`

Nota. La columna de prueba automatizada señala el archivo cuyo nombre y métodos corresponden al requisito; no se midió cobertura de código. `booking-flow.test.js` está en `lavarapido-infra/e2e` y necesita el sistema levantado.

6. Hallazgos y pendientes

Tabla 8

Brechas de prueba encontradas

#	Brecha	Recomendación
1	La app móvil no tiene pruebas automatizadas	Agregar Jest con `jest-expo` al menos para validadores (`shared/validators`) y servicios (`core/services`)
2	La mayoría de las 43 pruebas de la web solo verifican que el componente se crea	Probar más reglas reales: `authGuard` por rol, `authInterceptor` ante 401, formulario de reserva
3	Las 7 pruebas de persistencia de payment necesitan una base migrada y se omiten sin ella	Documentar en el README cómo crear la base de pruebas y definir `PAYMENT_INTEGRATION_DB`
4	De RF-023 (reportes) solo se prueban las métricas del operario y el resumen del equipo; la exportación PDF/Excel no tiene prueba	Agregar un caso funcional de reportes y exportación PDF/Excel
5	RF-032 (bahías) y RF-034 (cuentas de pago) no tienen pruebas automatizadas propias: `ScheduleService` y `PaymentAccountService` no tienen clase de prueba	Agregar pruebas unitarias de esos dos servicios
6	Los requisitos implementados después del 9 de octubre (RF-004b, RF-008, RF-010, RF-015, RF-022, RF-036, RF-037 y RF-038) tienen pruebas automatizadas pero no caso funcional propio; RF-008, RF-010, RF-015, RF-036 y RF-037 ya se recorren en las pruebas de punta a punta	Agregar sus casos CP y ejecutarlos contra el sistema completo

Tabla 9

Defectos encontrados al ejecutar el sistema

#	Defecto	Evidencia	Severidad
1	El dashboard del administrador saluda siempre "¡Hola, Laura!", sin importar quién inicia sesión	`Web/src/app/features/admin/pages/dashboard/dashboard.ts:73` (`adminName = 'Laura'`); también "Laura Méndez" fijo en `payment-review-modal.html:167`	Baja
2	En "Servicios asignados" del operario (web) la columna Cliente muestra la placa	`Web/src/app/features/operator/pages/assigned-services/assigned-services.ts:141`	Media
3	En el inicio del cliente (móvil) "Operario asignado" aparece vacío aunque la reserva tiene operario	Reserva RES-060003, asignada en CP-12; captura del móvil	Media
4	El nombre de la marca no es uniforme: "AquaWash" (móvil), "Car Wash" y "Lavado Vehicular" (web), "Express Car Wash" (establecimiento)	Capturas de la web y del móvil	Baja
5	La landing anuncia "Servicio móvil de lavado" y "donde estés", pero el SRS v4 define un lavadero de un único local	Captura de la landing; SRS v4, sección 1	Baja
6	Si hay dos cuentas del mismo método (dos Nequi), la pantalla de pagos las muestra marcadas a la vez	Captura de Pagos del cliente	Baja

> *Nota.* Revisión del código del 16 de octubre de 2026: los seis defectos están corregidos. El panel saluda con el nombre de la sesión y la revisión de pagos muestra a quien revisa (1); la columna Cliente muestra el nombre del cliente (2); el inicio del cliente muestra el operario asignado en la web y en el móvil (3); la marca es LavaRápido en la web y en el móvil (4); la página de inicio describe el servicio en el lavadero (5); y la pantalla de pagos selecciona por cuenta y muestra el titular cuando dos cuentas comparten medio (6, `payment.spec.ts`).

Referencias

International Organization for Standardization. (2021). *Software and systems engineering — Software testing — Part 3: Test documentation* (ISO/IEC/IEEE 29119-3:2021). https://www.iso.org/standard/79429.html

JUnit. (s.f.). *JUnit 5 user guide*. Recuperado el 9 de octubre de 2026, de https://junit.org/junit5/docs/current/user-guide/

Vitest. (s.f.). *Vitest documentation*. Recuperado el 9 de octubre de 2026, de https://vitest.dev/

xUnit.net. (s.f.). *xUnit.net documentation*. Recuperado el 9 de octubre de 2026, de https://xunit.net/